

Projet Fin SAS 1 Youcode: Gestion d'un train en console "Railway Manager"

Introduction

Vous devez développer une application console en **JavaScript avec Node.js** permettant de gérer les trajets et les tickets d'un train.

L'application fonctionne uniquement dans le terminal.

Les saisies utilisateur doivent être réalisées en utilisant **prompt()** ou le module natif **readline** de Node.js.

Exemple avec **readline** :

JavaScript

```
const readline = require('node:readline');

const { stdin: input, stdout: output } = require('node:process');

const rl = readline.createInterface({ input, output });

rl.question('What do you think of Node.js? ', (answer) => {

  console.log(`Thank you for your valuable feedback: ${answer}`);

  rl.close();

});
```

Exemple avec **prompt-sync** :

Il faut installer le package avec : **'npm install prompt-sync'**

JavaScript

```
var prompt = require('prompt-sync')();

var n = prompt('How many more times? ');
```

Les données nécessaires à la réalisation du projet sont déjà fournies. Elles contiennent la liste des trajets à utiliser dans l'application.

Vous devez utiliser ces données comme base de travail.

Données du projet: <https://gist.github.com/AzizBenMallouk/31e020b92fcc710a732fae0c2adec985>

1. Menu principal

Au lancement du programme, afficher :

```
JavaScript
=====
      RAILWAY MANAGER
=====

1. Afficher les trajets
2. Acheter un ticket
3. Afficher les tickets
4. Annuler un ticket
5. Rechercher un ticket
6. Filtrer les trajets
7. Trier les trajets
0. Quitter

Votre choix :
```

Après chaque opération, le programme doit revenir au menu principal.

Le programme s'arrête uniquement lorsque l'utilisateur choisit :

0. Quitter

2. Les trajets

Les trajets sont **fixes** et déjà enregistrés dans le programme.

Il n'est pas demandé de créer ou de calculer de nouveaux trajets.

Exemple :

JavaScript

```
const trips = [
  {
    id: 1,
    departure: "Safi",
    destination: "Yousseoufia",
    departureTime: "08:00",
    arrivalTime: "09:00",
    price: 25,
    availableSeats: 50
  },
  // Les autres données sont disponibles dans le fichier fourni.
];
```

Data : <https://gist.github.com/AzizBenMallouk/31e020b92fcc710a732fae0c2adec985>

Chaque trajet possède :

- un identifiant ;
- une ville de départ ;
- une destination ;
- une heure de départ ;
- une heure d'arrivée ;
- un prix ;
- un nombre de places disponibles.

3. Afficher les trajets

L'utilisateur doit pouvoir afficher tous les trajets disponibles.

Exemple :

JavaScript

=== TRAJETS DISPONIBLES ===

#1 Safi → Yousseoufia

Départ : 08:00

Arrivée : 09:00

Prix : 25 DH

Places disponibles : 50

#2 Yousseoufia → Ben Guerir

Départ : 09:30

Arrivée : 10:30

Prix : 30 DH

Places disponibles : 50

4. Acheter un ticket

L'utilisateur saisit :

Nom du passager :

Identifiant du trajet :

Exemple :

Nom du passager : Ahmed

Identifiant du trajet : 3

Le programme doit :

1. rechercher le trajet correspondant ;
2. vérifier que le trajet existe ;
3. vérifier qu'il reste au moins une place disponible ;
4. créer un ticket ;
5. attribuer automatiquement un numéro de place ;
6. diminuer le nombre de places disponibles ;
7. ajouter le ticket au tableau des tickets.

Les tickets sont stockés dans :

JavaScript

```
const tickets = [];
```

Exemple de ticket :

JavaScript

```
{  
  id: 1,  
  passengerName: "Ahmed",  
  tripId: 3,  
  seatNumber: 1,  
  price: 90  
}
```

```
}
```

L'identifiant du ticket doit être unique.

Si le trajet n'existe pas :

Trajet introuvable.

Si aucune place n'est disponible :

Train complet.

Sinon :

Exemple :

JavaScript

Ticket acheté avec succès.

Ticket #1

Passager : Ahmed

Trajet : Safi → Marrakech

Place : 1

Prix : 90 DH

5. Afficher les tickets

Le programme doit afficher tous les tickets enregistrés.

Exemple :

JavaScript

=== TICKETS ===

Ticket #1

Passager : Ahmed

Trajet : Safi → Marrakech

Place : 1

Prix : 90 DH

Ticket #2
Passager : Sara
Trajet : Safi → Youssoufia
Place : 1
Prix : 25 DH

Si aucun ticket n'existe :

Aucun ticket enregistré.

6. Annuler un ticket

L'utilisateur saisit : **Identifiant du ticket**

Le programme doit :

1. rechercher le ticket ;
2. vérifier qu'il existe ;
3. retrouver le trajet associé ;
4. supprimer le ticket ;
5. augmenter le nombre de places disponibles du trajet de 1.

Exemple :

```
JavaScript
Identifiant du ticket : 2

Ticket annulé avec succès.
```

Si le ticket n'existe pas :

Ticket introuvable.

7. Rechercher un ticket

L'application doit permettre de rechercher un ticket.

Recherche par nom du passager

Exemple :

JavaScript

Nom du passager : Ahmed

// Le programme affiche tous les tickets appartenant à Ahmed.

Ticket #3

Passager : Ahmed

Trajet : Marrakech → Casablanca

Place : 2

Prix : 120 DH

8. Filtrer les trajets

Les trajets étant déjà enregistrés, le programme doit simplement filtrer le tableau `trips`.

L'utilisateur peut faire un filtrer par ville de départ

JavaScript

Ville de départ : Safi

Résultat :

Safi → Youssoufia : 25 DH

Safi → Marrakech : 90 DH

9. Trier les trajets

L'utilisateur peut trier par **Prix croissant**

Exemple :

JavaScript

Safi → Youssoufia : 25 DH

Youssoufia → Ben Guerir : 30 DH

Safi → Marrakech : 90 DH

Marrakech → Casablanca : 120 DH

10. Bonus — Statistiques

Une fois toutes les fonctionnalités principales terminées, l'apprenant peut ajouter quelques statistiques.

Nombre total de tickets vendus

Nombre total de tickets : 15

Chiffre d'affaires total

Calculer la somme des prix des tickets.

Chiffre d'affaires total : 1 250 DH

Trajet le plus vendu

Compter le nombre de tickets correspondant à chaque `tripId`.

Exemple :

Trajet le plus vendu :

Safi → Marrakech

7 tickets vendus

11. Contraintes techniques

Le projet doit être réalisé en **JavaScript avec Node.js**.

Les notions principalement attendues sont :

- variables et constantes ;
- opérateurs ;
- `if / else` ;
- `switch` ;
- `for` ;
- `while` ;
- fonctions ;
- tableaux ;
- objets ;
- tableaux d'objets ;
- manipulation de chaînes de caractères.

Des méthodes JavaScript peuvent être utilisées, notamment :

`push()`
`splice()`
`find()`
`findIndex()`
`includes()`
`filter()`
`map()`

L'application ne doit pas utiliser :

- HTML ;
- CSS ;
- DOM ;
- navigateur ;
- framework frontend ;
- base de données.

Toutes les données restent en mémoire pendant l'exécution du programme.

12. Livrables

Vous devez fournir :

- le fichier JavaScript permettant d'exécuter l'application ;
- un dépôt GitHub ;
- un historique de commits montrant la progression du travail.